The propositions package*

Cian Dorr (with help from Claude Code)
ciandorr@gmail.com

2026/02/13

Abstract

The propositions package provides a key-value driven system for labelling propositions, theses, and premises in academic papers. Items may be given names like '(P)' or 'Physicalism', or auto-numbered using different counters; all carry robust cross-references with configurable formatting. The package integrates with amsmath, hyperref, and cleveref.

Contents

*This document describes version v0.2, dated 2026/02/13.

1 Introduction

In some academic disciplines (such as philosophy), it is common to have displayed propositions (examples, theses, premises,...) with various kinds of labels. A thesis might be referred to as '(P)' or 'Physicalism'; the premises of an argument might be numbered as 'P1', 'P2', 'P3', ...; or examples might be numbered consecutively over the course of a whole article. Standard LaTeX environments like `enumerate` can handle some of these cases, but cross-referencing is awkward: `\ref` produces a bare number or letter, and the author must manually add parentheses or other formatting at every point of reference. The standard `description` environment does not allow cross-referencing at all.

The propositions package solves this by attaching formatting information to each label. A short item like `\item[P]` (inside `prop`) is displayed as "(P)" and `\ref` automatically produces "(P)" as well—complete with parentheses, hyperlinks, and cleveref support. The full key-value interface supports named items, numbered items, custom counters, glosses, shorthands, and per-item format overrides.

The `\ptag` command (which requires amsmath) extends this to displayed math environments: an equation can be tagged with a proposition label instead of (or using) its equation number.

2 History

I wrote the ancestor to this package in the 90s while finishing my Ph.D. thesis, but never documented it or shared it with the world. This new version is a thorough re-implementation in LaTeX3, written in 2026 with extensive help from Claude Code. I hope others will find it as useful as I have.

3 Basic usage

Load the package with `\usepackage{propositions}` or `\usepackage[⟨options⟩]{propositions}` (see section 9 below for valid package options).

The `prop` environment generates a list of propositions, each introduced by `\item`. `\item` with an optional argument gives a `description`-like label:

```
\begin{prop}
  \item[Physicalism] Everything is
  physical. \label{phys}
  \item[Idealism] Everything is
  mental. \label{ideal}
\end{prop}
```

Physicalism Everything is physical.

Idealism Everything is mental.

Unlike the standard `description` environment, one can refer back to these propositions using the standard `\ref` command:

```
\ref{phys} is more plausible
than \ref{ideal}.
```

Physicalism is more plausible than Idealism.

With no optional argument, `\item` will by default generate numbered items similar to `enumerate`, but with numbering that persists across the document:

```
\begin{prop}
  \item Every atom is physical.
  \label{atoms}
\end{prop}
\ref{phys} follows from the
conjunction of \ref{atoms} and
\begin{prop}
  \item Everything is an atom.
  \label{atomism}
\end{prop}
```

(1) Every atom is physical.

Physicalism follows from the conjunction of (1) and

(2) Everything is an atom.

As with `enumerate`, the counter and formatting depend on the nesting level:

```
\begin{prop}
  \item \label{dual}
  \begin{prop}
    \item Some things are physical.
    \label{some}
    \item Some things are not
    physical. \label{notall}
  \end{prop}
\end{prop}
Without \ref{some}, \ref{dual}
would be consistent with \ref{ideal}.
```

(3) a. Some things are physical.

b. Some things are not physical.

Without (3a), (3) would be consistent with **Idealism**.

4 Advanced usage

The format of the proposition labels, and of subsequent references, are both configurable using a key=value syntax (see section 6 for the possible keys):

```
\begin{prop}
  \item[No Overlap,
    align=flush,
    display format=\textsc{#1},
    ref format=\textit{#1}]
    Nothing mental is
    physical. \label{incomp}
\end{prop}
Is \ref{incomp} consistent
with \ref{phys}?
```

4 Nothing mental is physical.

Is *4* consistent with **Physicalism**?

Preset *prop styles* can be declared and used instead of configuring the `\items` one at a time:

```
\begin{prop}
  \item[Nihilism,
    style=thesis]
    There is nothing.
    \label{nihilism}
\end{prop}
Does \ref{nihilism} imply
\ref{phys}, \ref{dual}, or
both?  Discuss.
```

5 There is nothing.

Does 5 imply **Physicalism**, (3), or both? Discuss.

Shortcut commands—e.g., `\titem[`⟨*keys*⟩`]` for `\item[style=thesis,` ⟨*keys*⟩`]` (within `prop`)—can also be defined. The package loads with a range of predefined styles.

When the package is loaded with `\usepackage[equations]{propositions}`, first-level `\item`s within `prop` use the same counter as equations. (This looks better with the `leqno` option to `\documentclass`.)

```
\begin{equation}
  \exists x (\text{Mental}(x)
  \wedge \text{Physical}(x))
\end{equation}
\begin{prop}
  \item
  There is overlap between
  the mental and the physical.
\end{prop}
```

(6) $\exists x(\text{Mental}(x) \wedge \text{Physical}(x))$

(7) There is overlap between the mental and the physical.

The `\ptag` command (requires amsmath) is an analogue of `\item` (within `prop`) that works inside displayed math environments.

```
\begin{equation}
  \ptag[Monism] \label{mon}
  \exists x \forall y(y = x)
\end{equation}
Is \ref{mon} compatible with
\ref{dual}?
```

Monism $\exists x \forall y(y = x)$

Is **Monism** compatible with (3)?

5 The `prop` environment and `\pitem`

`\begin{prop}[`⟨*keys*⟩`]`
⟨*environment content*⟩
`\end{prop}`

Creates a displayed list of propositions. It is a standard LaTeX list, so by default its formatting will depend on the standard length parameters like `\itemsep` and `\topsep`, although these can be overridden by setting keys.
Within `prop`, `\pitem` (see below) creates labelled items. The ordinary `\item` command is still available for unlabelled items.
Any display math environment (`\equation`, `align`, etc.) used inside `prop` or `inlineprop` automatically increments the nesting level for its duration, so that `\ptag` inside such an environment behaves as if it were one level deeper.
The optional ⟨*keys*⟩ argument accepts the same keys as `\propoptions` → P. 17 (section 6), with effects local to this environment.

`\begin{inlineprop}[`⟨*keys*⟩`]`
⟨*environment content*⟩
`\end{inlineprop}`

Like `prop`, but does not create a list. Allows `\pitem` to be used outside list environments, e.g. for generating numbers at the beginning of paragraphs. Steps the `prop` counter and increments the nesting level. Accepts the same optional ⟨*keys*⟩ as `prop`.

Within `prop` and `inlineprop`, `\item` is redefined to act as the proposition-item command. Its optional argument is a comma-separated list of ⟨*key*⟩=⟨*value*⟩ pairs (see section 6). A bare string without = is treated as a proposition name.

When used without an optional argument (or without setting `name`, `counter`, or `style`), the style is determined by the `nameless style` key (section 6). By default this is `numbered`, which dispatches by nesting depth to `levelone`, `leveltwo`, ... via `\proplevelchoice`.

`\pitem[`⟨*keys*⟩`]` (deprecated)

Deprecated alias for `\item` within `prop` or `inlineprop`. Issues a warning on each use. Replace with `\item`.

`\ptag[`⟨*keys*⟩`]`

(Available only when `amsmath` is loaded.) Works inside displayed math environments like `equation` and `align`. Accepts the same keys as `\item` within `prop`, except that `align` has no effect (since positioning is controlled by the tag placement system).

6 Keys

Most keys are available in three contexts, which follow one rule: an argument configures a single thing, while `\propoptions` → P. 17 sets a cascading default.

- In the optional argument of `\item` or `\ptag`: the key applies to that item only.
- In the optional argument of `prop` or `inlineprop`: the key applies to *that environment only*. A list-dimension key (subsection 6.2) sizes that list; an item-level key acts as a default for that environment's own `\item`s. Neither reaches a nested `prop`: an argument does not cascade.
- Via `\propoptions` → P. 17 or as a `\usepackage` option: the key sets a default for *every subsequent* `prop`/`inlineprop`, nested ones included. A list-dimension key applies to each such list; an item-level key (subsection 6.1) is a default before each `\item` at every level. The effect is group-scoped (`\begingroup \propoptions{...} \endgroup` scopes it as usual).

So to style a whole tree of nested environments, use `\propoptions` → P. 17; to style just one environment, use its argument. When the same key is set several ways, precedence is: explicit item key > environment argument > `\propoptions` → P. 17 > style preset. The `equations` key is global only (subsection 6.6).

6.1 Item-level keys

`name=`⟨*text*⟩ (no default)

The proposition's name. A bare string (without =) in the key list is equivalent to `name=`⟨*text*⟩.

`style=`⟨*style*⟩ (no default)

A prop style, equivalent to a preset collection of keys. Styles can be declared using `\SetPropStyle` → P. 10 or `\DeclareNumberedStyle` → P. 11, and several come predefined (section 7).

`counter=`⟨*name*⟩ (no default)

Counter to use. The counter is automatically stepped, and the item's `name` is set to `\the`⟨*name*⟩ (though this can be overridden by `counter format` or `name`).

`counter format=`⟨*template*⟩ (no default)

How to display the counter value. Use `#1` for the counter name, e.g. `counter format=\roman{#1}`. When set, this is used instead of `\the`⟨*counter*⟩ for generating the item's name.

`format=`⟨*template*⟩ (no default)

Formatting applied to the `name`: use `#1` for the argument, e.g. `format=\textbf{(#1)}`. Shorthand for setting both `display format` and `ref format`.

`display format=`⟨*template*⟩ (no default)

Format for displaying the name in the proposition's label. Does not affect cross-references.

`label format=`⟨*template*⟩ (no default)

Format applied to the *entire* assembled label, i.e. the name (after `display format`), shorthand, and gloss together. Use `#1` for the whole assembled text. For example, `label format=#1:` appends a colon after the complete label. Does not affect cross-references.

`ref format=`⟨*template*⟩ (no default)

Format for subsequent cross-references to this proposition. Does not affect the display.

`shorthand=`⟨*text*⟩ (no default)

An abbreviation displayed after the name. If present, the shorthand becomes the reference text: `\ref` produces the shorthand (formatted with `ref format`) rather than the full name.

`shorthand format=`⟨*template*⟩ (initially `~[#1]`)

Format for displaying the shorthand in the label.

`gloss=`⟨*text*⟩ (no default)

A parenthetical gloss displayed after the name. Does not affect cross-references.

`gloss format=`⟨*template*⟩ (initially `~(#1)`)

Format for displaying the gloss in the label.

`ref=`⟨*text*⟩ (no default)

Explicitly set the reference text, overriding what would be derived from `name`, `counter`, or `shorthand`.

`label=`⟨*label*⟩ (no default)

Equivalent to a trailing `\label{`⟨*label*⟩`}`.

`crefname=`⟨*type*⟩ (no default)

When cleveref is loaded, assigns an arbitrary reference type to this proposition. For example, `crefname=lemma` causes `\cref` to use the names defined by `\crefname{lemma}{...}{...}` instead of the default `proposition` type. The ⟨*type*⟩ must be known to cleveref; new types can be declared with `\crefname`.

`reset=`⟨*boolean*⟩ (initially `true`)

When `true` (the default), processing this `\pitem` resets the sub-level counter one nesting depth below the current level (e.g. `numpropii` at level 1). Set `reset=false` to suppress this reset, so that sub-item numbering continues from where it left off across consecutive parent items. Can also be set in a style definition via `\SetPropStyle` → P. 10.

`align=`⟨*value*⟩ (no default)

How the label should be positioned. Possible values: `left` (standard left-aligned label, offset controlled by `\labelwidth` and `\labelsep`), `right` (right-aligned, like `enumerate`), `center` (centered within the label box), `flush` (aligned with left margin of item text), `nextline` (label on its own line), and `flush-nextline`. Has no effect inside `inlineprop` or `\ptag`. For left-margin alignment, use `labelindent=0em` instead.

6.2 List dimensions

These keys are effective in all three contexts. At item level, a dimension key applies only to the label of that specific item; at environment and global level it applies to the whole list.

`topsep=`⟨*length / length list*⟩
`partopsep=`⟨*length / length list*⟩
`itemsep=`⟨*length / length list*⟩
`parsep=`⟨*length / length list*⟩
`leftmargin=`⟨*length / length list*⟩
`rightmargin=`⟨*length / length list*⟩
`labelwidth=`⟨*length / length list*⟩
`labelsep=`⟨*length / length list*⟩
`itemindent=`⟨*length / length list*⟩
`listparindent=`⟨*length / length list*⟩

Override the standard LaTeX list dimensions. Accept the same values as `\setlength`, including rubber lengths (e.g. `itemsep=4pt plus 2pt`). Each key may also take a comma-separated list of per-level values: e.g. `leftmargin={2.5em, 0em}`. The first value applies at level 1, the second at level 2, etc. Gaps in the list (e.g. `leftmargin={, 0em}`) cause the class default to be used for that level.

`labelindent=`⟨*length / length list*⟩ (no default)

Positions the left edge of the label box at ⟨*length*⟩ from the enclosing margin, adjusting `\labelwidth`, `\leftmargin`, `\labelsep`, or `\itemindent` as needed. (Since the value of any one of these dimensions can be derived from those of the other four, it never makes sense to set all five.)

`tightspacing` (no value)

Sets all vertical spacing to the compact defaults that the standard document classes use for level-three lists (`topsep` and `itemsep` to `2pt` with stretch/shrink, `parsep` to `0pt`, `partopsep` to `1pt`). Individual dimension keys set afterward override.

`nosep` (no value)

Sets `\topsep`, `\itemsep`, and `\parsep` all to zero.

`continue=`⟨*boolean*⟩ (initially `true`)

When `true`, a `prop` environment that immediately follows another `prop` (with no intervening paragraph text) replaces the normal `\topsep`-based inter-list space with `\itemsep` + `\parsep`, giving the visual appearance of a single continuous list.

`items=`⟨n⟩ (initially `1`)

Indicates that approximately ⟨n⟩ items are expected in this environment. Used together with `\propformlabel` to compute a preview label wide enough for the ⟨n⟩th expected item, so that dimension expressions such as `leftmargin=\widthof{\propformlabel}+0.5em` reserve enough space for the widest label. Has no effect on actual item processing or counter stepping.

`\propformlabel`

Expands to the formatted label of the (approximately) `items`-th item in this environment, computed at `\begin{prop}` *before* the list dimensions are applied, so it can be used directly in dimension expressions. Updated after each `\item` to reflect the most recently output label. Typical use:

```
%   \begin{prop}[items=20, leftmargin=\widthof{\propformlabel}+0.5em]
%
```

sizes the left margin to accommodate labels up to the 20th item.

6.3 Wrapping the list in another environment

`wrapper begin=`⟨*code*⟩
`wrapper end=`⟨*code*⟩

Insert arbitrary ⟨*code*⟩ immediately before `\begin{list}` and immediately after `\end{list}`, so that the whole list is wrapped in another environment. These keys act at the environment level: they fire once around the entire list, not once per `\item`. They follow the usual rule (section 6): set in a `prop` argument (or its `style=`) a wrapper frames *that one list* and does not reach a nested `prop`; set via `\propoptions` → P. 17 it frames *every* subsequent list, nested ones included. So a `framed` sublist inside a `framed` list nests only when the sublist asks for it; and to frame a whole tree of lists at once, use `\propoptions` → P. 17. At item level the keys are ignored.

The package provides a ready-made `framed` style (subsection 7.1) that frames the list with tcolorbox (which must be loaded). To frame a list, just write `\begin{prop}[style=framed]`.

To build your own wrapper style, set `wrapper begin` / `wrapper end` directly. Brace any value that contains a comma (such as mdframed options), so that the comma is not read as a key separator:

```
\SetPropStyle{redbox}{
  wrapper begin = {\begin{mdframed}[linecolor=red, linewidth=2pt]},
  wrapper end   = {\end{mdframed}}}
```

Because a wrapper style is an ordinary style, another style can inherit the frame with `style = framed` and add its own formatting on top. The wrapping environment must be one that can be split into separate begin and end code—that is, it must not read its body as a macro argument. mdframed and the default tcolorbox environment qualify; environments that grab their body (via `\collect@body` or similar) do not. To turn a wrapper off again within a group, set the keys to empty (`wrapper begin = {}, wrapper end = {}`).

The built-in `framed` style in action:

```
\begin{prop}[style=framed]
  \item A framed proposition.
  \item And a second one.
\end{prop}
```

(8) A framed proposition.
(9) And a second one.

See (8) and (9).

`\propoperativeleftmargin`

A read-only length giving the `leftmargin` an ordinary (unframed) `prop` would use at the current nesting level—the ambient `\propoptions` → P. 17/class value, resolved per level. It is recomputed at every `\begin{prop}`, so it is valid inside a wrapper (before the environment's own `leftmargin` has taken effect), which is how the built-in `framed` style positions its frame to match an unframed list.

6.4 Default styles

When `\pitem` or `\ptag` is used without an explicit `style`, one of the following defaults is selected based on whether a name/counter was given and whether ptag mode is active.

`named style=`⟨*style*⟩ (initially `proposition`)

The style used when a `name` or `counter` is given but no explicit `style`.

`named ptag style=`⟨*style*⟩ (initially empty)

If set, overrides `named style` for `\ptag` items only.

`nameless style=`⟨*style*⟩ (initially `numbered`)

The style used when `\pitem` or `\ptag` has no optional argument (no name, counter, or style).

`nameless ptag style=`⟨*style*⟩ (initially empty)

If set, overrides `nameless style` for `\ptag` items only.

6.5 Environment-level style and item defaults

Any key from subsection 6.1 may appear in the optional argument of `prop` or `inlineprop`. It is then applied as a default before each `\item` of *that* environment that does not set the key explicitly. An item-level key always overrides the environment default, and the default does not reach the `\item`s of a nested environment (an argument styles only its own environment; use `\propoptions` → P. 17 for a cascading default).

Important ordering note. Style selection (which decides whether an item is "named" or "nameless") is based solely on the item's *own* optional argument, before environment defaults are applied. Consequently, `\begin{prop}[name=foo]\item bar` produces a *nameless*-style item with label `(foo)`: the environment default `name=foo` is not seen during style resolution. To get the equivalent of `\item[name=foo]` (named style, bold), use `\begin{prop}[style=proposition, name=foo]\item bar`.

`style=`⟨*style*⟩ (no default)

Sets a default style for all `\item`s in this environment. The style's list-dimension keys (`leftmargin`, `labelwidth`, etc.) size this list; its item-level keys are applied as defaults for each of this environment's `\item`s. An explicit `style=` on an individual `\item` overrides this default, and (like any argument key) the style does not cascade into a nested `prop`. This key is also accepted by `\propoptions`, where it *does* cascade: it becomes the default style for all subsequent environments at every level (group-scoped, overridden by the env arg, and overridden by per-item `style=`). So `\begin{prop}[style=framed]` frames one list, whereas `\propoptions{style=framed}` frames every following list.

6.6 Equation numbering

`equations` (no value, **global only**)

Sets `nameless style=eqnum` and installs hooks so that the LaTeX and amstex displayed equation environments (such as `equation` and `align`) use the same formatting as the `equation` prop style. The `equation` style's `display format` and `ref format` keys are used for generating equation numbers and cross-references to them. Redefining the `equation` style, e.g. with

```
\SetPropStyle{equation}{format = [#1]}
```

will automatically update the hooks.

7 Prop styles

`\SetPropStyle{`⟨*name*⟩`}{`⟨*keys*⟩`}`

Defines or modifies a prop style for use with the `style` key. All `\pitem` → P. 5 keys are accepted, plus the following:

`style=`⟨*parent*⟩ (no default)

Inherit from a parent style. When an item is created, the parent's settings are loaded first (recursively, if the parent itself has a parent), then this style's own keys are applied on top.

The argument may be a macro which is expanded when the item is created: for example, a style with `style=\{`⟨*style1*⟩`},{`⟨*style2*⟩`},{`⟨*style3*⟩`}` will resolve to `{`⟨*style1*⟩`}`, `{`⟨*style2*⟩`}`, `{`⟨*style2*⟩`}` depending on the nesting depth. The built-in `numbered` and `eqnum` styles use this mechanism.

`macro=`⟨*command*⟩ (no default)

A new user macro, equivalent to `\pitem[style=`⟨*name*⟩`]`. Any further keys given to the macro are passed to `\pitem`.

If the style ⟨*name*⟩ already exists, `\SetPropStyle` modifies or adds keys. For example, `\SetPropStyle{proposition}{align=flush}` changes the alignment of the built-in `proposition` style while preserving its other settings.

```
\SetPropStyle{angle}{
  labelindent = 0em,
  display format =
    \textbf{$\langle$#1$\rangle$},
  ref format     =
    $\langle$#1$\rangle$,
  macro = \angitem
}
\begin{prop}
  \angitem[Angle thesis]
  Everything is angular.
\end{prop}
No further discussion of
\lastref{} is needed.
```

⟨**Angle thesis**⟩ Everything is angular.

No further discussion of ⟨Angle thesis⟩ is needed.

`\DeclareNumberedStyle{`⟨*name*⟩`}[`⟨*keys*⟩`]`

Creates a new LaTeX counter named ⟨*name*⟩ and a matching prop style with `counter=`⟨*name*⟩. All `\SetPropStyle` → P. 10 keys are accepted, plus:

`parent=`⟨*counter*⟩ (no default)

A parent counter; the new counter resets when the parent steps (same mechanism as `\numberwithin`). A dedicated `prop` counter (stepped by each `prop` and `inlineprop`) is available for non-persistent numbering.

```
\DeclareNumberedStyle{P}
\begin{prop}
  \item[counter=P] First premise.
  \label{p1}
  \item[counter=P] Second premise.
  \label{p2}
\end{prop}
From \ref{p1} and \ref{p2}\ldots
```

P1 First premise.

P2 Second premise.

From **P1** and **P2**...

`\proplevelchoice{`⟨*item1, item2, ...*⟩`}`

Picks the entry at the current nesting depth (1-indexed, clamped to the list length). At depth 0 (outside any `prop` environment), returns the first entry. This macro is fully expandable and can be used in the `name`, `format`, and `style` keys of `\SetPropStyle` to create depth-dependent styles.

7.1 Built-in styles

The following prop styles are predefined. Each entry shows the defining code (using user-facing commands) and a live example. All styles can be modified with `\SetPropStyle` → P. 10.

Text styles

`plain` Unformatted text label.

```
\SetPropStyle{plain}{format = #1}
```

Claim An unformatted item.

`proposition` Bold text label. Auto-selected when `\item` has a name but no `style`.

```
\SetPropStyle{proposition}{format = \textbf{#1}}
```

Thesis A bold-formatted item.

`thesis` Small-caps name at the text margin (`flush` alignment). Shortcut: `\titem`.

```
\SetPropStyle{thesis}{format = \textsc{#1},
  align = flush, macro = \titem}
```

Hypothesis A flush-aligned item.

`vignette` Italic name at the text margin with a colon in the label.

```
\SetPropStyle{vignette}{ref format = \textit{#1},
  align = flush, label format = \textit{#1:}}
```

Example: A vignette item.

`bullet` Bullet symbol varying by depth (like `\itemize`). Shortcut: `\bitem`.

```
\SetPropStyle{bullet}{align = center,
  name = \proplevelchoice{\textbullet,
    {\normalfont\textendash}, \textasteriskcentered,
    \textperiodcentered},
  display format = #1, macro = \bitem}
```

- A bullet item.

Numbered styles with dedicated counters

`roman` Roman-numeral counter, resets per section. Shortcut: `\ritem`.

```
\DeclareNumberedStyle{roman}[parent = section,
  counter format = \roman{#1}, format = (#1), macro = \ritem]
```

(i) A roman-numbered item.

`alph` Alphabetic counter, resets per section. Shortcut: `\aitem`.

```
\DeclareNumberedStyle{alph}[parent = section,
  counter format = \alph{#1}, format = (#1), macro = \aitem]
```

(a) An alphabetic item.

Level-specific styles

These styles are used by the `numbered` and `eqnum` dispatcher styles (below). They share the counters `numpropi` – `numpropv`, which reset automatically when an `\item` at the next lower level is processed.

`levelone` Level-1 numbered items: arabic in parentheses.

```
\SetPropStyle{levelone}{counter = numpropi, format = (#1)}
```

`leveltwo` Level-2: lowercase alphabetic with dot.

```
\SetPropStyle{leveltwo}{counter = numpropii,
  display format = #1., ref format = \parentref{#1}}
```

`levelthree` Level-3: lowercase roman in parentheses.

```
\SetPropStyle{levelthree}{counter = numpropiii,
  display format = (#1), ref format = \parentref{.#1}}
```

`levelfour` Level-4: uppercase alphabetic with dot.

```
\SetPropStyle{levelfour}{counter = numpropiv,
  display format = #1., ref format = \parentref{#1}}
```

`levelfive` Level-5: uppercase roman in parentheses.

```
\SetPropStyle{levelfive}{counter = numpropv,
  display format = (#1), ref format = \parentref{.#1}}
```

`equation` Uses the `equation` counter. When the `equations` option is active, `\SetPropStyle{equation}{...}` automatically updates the `\tagform@` and `\ref` hooks for all equations.

```
\SetPropStyle{equation}{counter = equation, format = (#1)}
```

Dispatcher styles

These styles use `style` with `\proplevelchoice` to dispatch to a level-specific style at resolution time.

`numbered` The default nameless style. Dispatches to `levelone` – `levelfive` by nesting depth.

```
\SetPropStyle{numbered}{style = \proplevelchoice{
  levelone, leveltwo, levelthree, levelfour, levelfive}}
```

`eqnum` Like `numbered`, but uses `equation` at level 1. Activated by the `equations` package option (which sets `nameless style = eqnum`).

```
\SetPropStyle{eqnum}{style = \proplevelchoice{
  equation, leveltwo, levelthree, levelfour, levelfive}}
```

`hierarchical` Produces 1, 1.1, 1.1.1 … numbering using `\parentref` and `counter format`. Defined via two helper styles:

```
\SetPropStyle{h-base}{counter = numpropi,
  counter format = \arabic{#1}, format = #1}
\SetPropStyle{h-sub}{
  counter = \proplevelchoice{numpropi, numpropii,
    numpropiii, numpropiv, numpropv},
  counter format = \arabic{#1},
  format = \parentref{.#1}}
\SetPropStyle{hierarchical}{style = \proplevelchoice{
  h-base, h-sub}}
```

Framed style

`framed` Frames the whole list in a tcolorbox (which must be loaded). Items are set `flush` and the list runs at `leftmargin=0` inside the frame, so the contents keep the current level's effective left margin (read via `\propoperativeleftmargin` → P. 9, so it honours any `\propoptions` → P. 17 `leftmargin`) while the frame is centred with its rule one `\propframepad` (default 6 pt) to the left of them. The whole geometry lives in the style definition, so it is easy to copy and adapt; and being an ordinary style it can be inherited, `\SetPropStyle{myframe}{style=framed, format=\textbf{#1}}`. Change the padding with `\setlength\propframepad{...}`.

Note A framed list, with a flush label and a plain paragraph.
The unlabelled line sits flush at the same edge.

8 Cross-referencing

Labels placed after `\item` items (within `prop`) work with the standard `\label`/`\ref` mechanism. The key difference from ordinary LaTeX references is that `\ref` produces *formatted* output: for example, `\textbf` might be applied to the name, or the number might be wrapped in parentheses. The formatting is controlled by the `format` key (or separately by `display format` and `ref format`).

`\Ref{`⟨*label*⟩`}`
`\Ref*{`⟨*label*⟩`}`

Titlecase variants of `\ref` and `\ref*`: uppercases the first letter of the formatted output. (For example, if `\ref{thesis}` produces 'the Identity Theory', then `\Ref{thesis}` produces 'The Identity Theory'.) The starred form suppresses the hyperlink. Note: `\Ref` only affects references produced by this package (which are stored via `\propapply`); for other references, it behaves like `\ref`.

`\nref{`⟨*label*⟩`}`
`\nref*{`⟨*label*⟩`}`
`\Nref{`⟨*label*⟩`}`
`\Nref*{`⟨*label*⟩`}`

"Naked ref." Outputs the bare reference content with all formatting stripped. If `\ref{premise}` produces '(P1)', then `\nref{premise}` produces 'P1'. The starred form suppresses the hyperlink. `\Nref` is the titlecase variant: it uppercases the first letter of the bare content.

`\nref` can be useful in the argument of `\item`, when the the name of one proposition should depend on that of another:

```
\SetPropStyle{proposition}{format=(#1)}
\begin{prop}
  \item[Phys]
  Everything is physical.
  \label{phys2}
  \item[\nref{phys2}*]
  Almost everything is physical.
  \label{newphys2}
  \item[\ref{phys2}*]
  This one has two sets of parentheses,
  which is probably not desired.
  \ref{newphys2} avoided this by using
  |\nref|.
\end{prop}
```

(Phys) Everything is physical.

(Phys*) Almost everything is physical.

((Phys)*) This one has two sets of parentheses, which is probably not desired. (Phys*) avoided this by using `\nref`.

Warning: documents where the name of one item includes a reference to that of another, and there are further references to that item, will require multiple LaTeX runs to resolve all references. To save time, it is better to avoid long chains of dependencies of this sort.

`\oref[`⟨*prefix*⟩`][`⟨*suffix*⟩`]{`⟨*label*⟩`}`
`\oref*[`⟨*prefix*⟩`][`⟨*suffix*⟩`]{`⟨*label*⟩`}`
`\Oref[`⟨*prefix*⟩`][`⟨*suffix*⟩`]{`⟨*label*⟩`}`
`\Oref*[`⟨*prefix*⟩`][`⟨*suffix*⟩`]{`⟨*label*⟩`}`

"Ref with options." Extends `\ref` by injecting a prefix and/or suffix *inside* the formatting. With one optional argument, ⟨*suffix*⟩ is appended; with two, ⟨*prefix*⟩ is prepended and ⟨*suffix*⟩ appended. For instance, if `\ref{premise}` produces '(P1)', then `\oref[*]{premise}` produces '(P1*)' and `\oref[old~][*]{premise}` produces '(old~P1*)'.
`\Oref` is the titlecase variant; the starred forms suppress the hyperlink.

`\oref` can also be useful in the name of `\item`s, if one wants the display format for the modified item to depend on that originally used

```
\begin{prop}
  \label{premise}
  \item[style=plain,name=\oref[*]{phys2}]
  This will use boldface and
  parentheses because the original
  referenced item did.
\end{prop}
```

(Phys*) This will use boldface and parentheses because the original referenced item did.

Another handy use for `\oref` is in combination with `\nref` to refer to ranges:

```
The first two numbered examples in
this document were
were \oref[--\nref{atomism}]{atoms}.
```

The first two numbered examples in this document were were (1–2).

`\lastref[`⟨*prefix*⟩`]{`⟨*suffix*⟩`}`
`\Lastref[`⟨*prefix*⟩`]{`⟨*suffix*⟩`}`

Produces a reference (witth no hyperlink) to the most recently processed `\pitem` or `\ptag`, even without a `\label`. Useful for back-references in running text. With one argument, ⟨*suffix*⟩ is appended; with two, ⟨*prefix*⟩ is also prepended. Use `\lastref{}` for a plain reference. `\Lastref` is the titlecase variant.

`\nlastref`
`\nLastref`

Like `\lastref{}`, but returns the bare content without formatting. `\nLastref` is the titlecase variant.

`\parentref[`⟨*prefix*⟩`]{`⟨*suffix*⟩`}`
`\Parentref[`⟨*prefix*⟩`]{`⟨*suffix*⟩`}`

Inside a nested `prop` (or `inlineprop`), produces a formatted reference to the most recent item of the enclosing level. Same argument convention as `\lastref`. `\Parentref` is the titlecase variant.

`\nparentref`
`\nParentref`

Like `\parentref{}` but returns the bare content without formatting. Takes no arguments; simply output any desired suffix directly afterwards. `\nParentref` is the titlecase variant.

`\parentref` and `\nparentref` are useful for making subitems whose names derive from their parent's:

```
\DeclareNumberedStyle{inner}[
  counter format=\alph{inner},
  display format=#1.]
\begin{prop}
  \item[OI] Outer item.
  \label{outer2}
  \begin{prop}
    \item[style=inner,
      format=\parentref{.#1}]
    \label{dsub1}
    Ref: \ref{dsub1},
    naked: \nref{dsub1}.
    \item[style=inner,
      display format=#1.,
      ref format=\parentref{.#1}]
    \label{dsub2}
    Ref: \ref{dsub2},
    naked: \nref{dsub2}.
  \end{prop}
\end{prop}
```

OI Outer item.
OI.a Ref: **OI.a**, naked: a.
b. Ref: **OI.b**, naked: b.

The built-in `leveltwo` and `levelthree` styles have `ref format=\parentref{#1}` and `ref format=\parentref{.#1}`, respectively, so that if the parent references as '(**P1**)', a `leveltwo` sub-item references as '(**P1a**)' and `\nref` returns just 'a'.

8.1 How it works: \propapply

`\propapply{`⟨*template*⟩`}{`⟨*content*⟩`}`

Internally, each reference is stored in the `.aux` file as `\propapply{`⟨*template*⟩`}{`⟨*content*⟩`}`. The ⟨*template*⟩ contains formatting with the placeholder `\propfmtarg` where content appears. At reference time, `\propapply` evaluates the template with `\propfmtarg` bound to ⟨*content*⟩. The `\oref` →P. 15 and `\nref` →P. 15 commands work by locally redefining `\propapply`.
In normal use, you need not interact with `\propapply` directly.

`\propfmtarg`

Placeholder used inside templates; expands to the content argument of the enclosing `\propapply`.

9 Package options

`\propoptions{`⟨*keys*⟩`}`

Sets cascading defaults (see section 6). Every key—list dimensions and item-level keys (subsection 6.1) alike, including `style=`—becomes a default for *every subsequent* `prop`/`inlineprop`, nested ones included, until overridden by an environment argument or an explicit item key. In particular, `\propoptions{style=thesis}` makes every subsequent `prop` (at any depth) use `thesis` by default. The effect is group-scoped: `\begingroup \propoptions{style=thesis} ..` applies the default only within the group. This is the counterpart to an environment argument, which sets the same keys for a *single* environment with-

out cascading (subsection 6.5); package options behave like a preamble-level `\propoptions`.

10 Compatibility

The propositions package is designed to work with hyperref, cleveref, and amsmath. amsmath is required for `\ptag` and the `equations` option. With cleveref loaded, all proposition items are assigned to a default `proposition` "reference type"; the `crefname` key can override this.

Recommended load order:

```
\usepackage{hyperref}
\usepackage{amsmath}   % if using \ptag
\usepackage{propositions}
\usepackage{cleveref}  % if used
```

11 Known issues

When using `\ptag` with a named counter (e.g. `\ptag[counter=P]`) inside an amsmath equation environment, hyperref may emit warnings of the form:

```
pdfTeX warning: destination with the same
identifier (name{equation.N}) has been already
used, duplicate ignored
```

These warnings are harmless and do not affect the correctness of cross-references.